UTS

PEMBELAJARAN MESIN



NAMA : Nurfadillah Aprilia Putri

NIM : 202431126

KELAS : G

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : 22

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Morgan Immanuel Nainggolan

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I **3**
PENDAHULUAN **3**
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II **4**
LANDASAN TEORI **4**
2.1 Machine Learning 4
2.2 Klasifikasi (Classification) 4
2.3 Dataset 4
2.4 Data Preprocessing 5
2.4.1 Pemeriksaan Missing Value 5
2.4.2 Penghapusan Data Duplikat 5
2.4.3 One-Hot Encoding 5
2.4.4 Standardisasi Data 5
2.5 Pembagian Data Training dan Testing 6
2.6 Algoritma K-Nearest Neighbors (KNN) 6
2.6.1 Euclidean Distance 6
2.6.2 Penentuan Nilai K 7
2.7 Algoritma Naive Bayes 7
2.7.1 Gaussian Naive Bayes 7
2.8 Cross Validation 8
2.9 Evaluasi Model Klasifikasi 8
2.9.1 Accuracy 8
2.9.2 Precision 8
2.9.3 Recall 8
2.9.4 F1-Score 9
2.10 Confusion Matrix 9
2.11 Python dan Scikit-Learn 9
BAB III **10**
HASIL **10**
3.1 Persiapan dan Pembacaan Dataset 10
3.2 Pemeriksaan dan Visualisasi Data 13
3.3 Preprocessing Data 17
3.4 Penerapan Algoritma K-Nearest Neighbors (KNN) 19
3.5 Penerapan Algoritma Naive Bayes 23
3.6 Perbandingan dan Evaluasi Hasil Model 23
3.7 Pembahasan Hasil Praktikum 25
BAB IV **26**
PENUTUP **26**
4.1 Kesimpulan Landasan Teori 26
4.2 Kesimpulan Hasil Praktikum 26
DAFTAR PUSTAKA **27**

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Berdasarkan praktikum UTS yang telah dilakukan, terdapat beberapa rumusan masalah sebagai berikut:

1. Bagaimana cara melakukan pengolahan dan preprocessing dataset Counter Strike 2 agar dapat digunakan dalam proses klasifikasi?
2. Bagaimana penerapan algoritma K-Nearest Neighbors (KNN) dan Naive Bayes untuk memprediksi tim yang memenangkan ronde?
3. Bagaimana cara mengevaluasi hasil klasifikasi menggunakan accuracy, precision, recall, dan F1-score?
4. Algoritma manakah yang memberikan hasil akurasi terbaik berdasarkan pengujian yang telah dilakukan?

1.2 Tujuan Masalah

Adapun tujuan dari pelaksanaan praktikum UTS ini adalah sebagai berikut:

1. Memahami proses pengolahan dan preprocessing dataset menggunakan Python.
2. Menerapkan algoritma K-Nearest Neighbors (KNN) dan Naive Bayes dalam proses klasifikasi.
3. Mengetahui cara melakukan pengujian dan evaluasi model berdasarkan hasil prediksi.
4. Membandingkan performa kedua algoritma untuk mengetahui model dengan tingkat akurasi terbaik.

1.3 Manfaat Masalah

Praktikum ini memberikan manfaat dalam meningkatkan pemahaman mengenai penerapan Machine Learning, khususnya algoritma KNN dan Naive Bayes. Selain itu, praktikum ini juga menambah keterampilan dalam mengolah dataset, membangun model klasifikasi, serta mengevaluasi hasil prediksi menggunakan Python. Pengetahuan tersebut dapat menjadi bekal untuk menyelesaikan permasalahan klasifikasi pada dataset lainnya.

BAB II
LANDASAN TEORI

2.1 Machine Learning

Machine Learning merupakan bagian dari kecerdasan buatan (Artificial Intelligence) yang memungkinkan komputer mempelajari pola dari suatu data untuk menghasilkan prediksi atau keputusan. Machine Learning dapat digunakan untuk mengolah data, mengenali pola, mengelompokkan informasi, serta memperkirakan suatu hasil berdasarkan data yang tersedia.

Salah satu penerapan Machine Learning adalah klasifikasi, yaitu proses menentukan kategori suatu data berdasarkan karakteristik yang dimilikinya. Dalam praktikum ini digunakan pendekatan Supervised Learning karena model dilatih menggunakan dataset yang sudah memiliki label atau target. Algoritma yang digunakan adalah K-Nearest Neighbors (KNN) dan Naive Bayes untuk memprediksi tim yang memenangkan ronde dalam permainan Counter Strike 2.

2.2 Klasifikasi (Classification)

Klasifikasi merupakan teknik dalam Machine Learning yang digunakan untuk mengelompokkan data ke dalam kategori tertentu berdasarkan pola yang ditemukan selama proses pelatihan. Proses ini menggunakan data yang sudah memiliki label sebagai bahan pembelajaran sehingga model dapat mengenali hubungan antara fitur dan target.

Dalam praktikum ini, klasifikasi digunakan untuk memprediksi hasil kemenangan ronde pada permainan Counter Strike 2. Fitur yang digunakan meliputi waktu pertandingan, skor tim, kesehatan pemain, armor, jumlah uang, status bom, dan map yang digunakan. Sementara itu, kolom round_winner digunakan sebagai target klasifikasi yang terdiri dari dua kategori, yaitu CT dan T.

2.3 Dataset

Dataset merupakan kumpulan data yang disusun dalam bentuk baris dan kolom. Setiap baris menunjukkan satu observasi, sedangkan setiap kolom berisi atribut atau informasi tertentu yang digunakan dalam proses analisis.

Pada praktikum ini digunakan dataset Counter Strike 2 yang memiliki 2.000 baris dan 14 kolom sebelum dilakukan pembersihan data. Dataset berisi informasi mengenai kondisi pertandingan, seperti skor tim CT dan T, waktu tersisa, kesehatan, armor, jumlah uang, map, serta hasil kemenangan ronde.

Dataset tersebut digunakan sebagai bahan untuk melatih dan menguji model klasifikasi agar dapat mengenali pola hubungan antara kondisi pertandingan dengan hasil kemenangan salah satu tim.

2.4 Data Preprocessing

Data preprocessing merupakan tahapan pengolahan awal yang dilakukan untuk mempersiapkan dataset sebelum digunakan dalam proses Machine Learning. Tahapan ini bertujuan untuk memastikan data memiliki format yang sesuai dan dapat diproses oleh algoritma yang digunakan. Cho et al. (2022) menjelaskan bahwa pemilihan teknik preprocessing dapat memengaruhi performa model klasifikasi.

Beberapa tahapan preprocessing yang digunakan dalam praktikum ini adalah sebagai berikut.

2.4.1 Pemeriksaan Missing Value

Missing value merupakan kondisi ketika terdapat nilai kosong pada suatu kolom dalam dataset. Pemeriksaan dilakukan untuk mengetahui apakah terdapat informasi yang belum terisi. Pada praktikum ini digunakan fungsi isnull().sum() untuk menghitung jumlah data kosong pada setiap kolom. Hasil pemeriksaan menunjukkan bahwa tidak ditemukan missing value pada dataset.

2.4.2 Penghapusan Data Duplikat

Data duplikat merupakan data yang memiliki isi sama dan muncul lebih dari satu kali. Data tersebut perlu diperiksa agar tidak menyebabkan informasi yang sama dihitung berulang. Penghapusan dilakukan menggunakan fungsi drop_duplicates(). Pada praktikum ini ditemukan 39 data duplikat sehingga jumlah dataset berkurang dari 2.000 menjadi 1.961 baris.

2.4.3 One-Hot Encoding

One-Hot Encoding merupakan teknik untuk mengubah data kategorikal menjadi bentuk numerik agar dapat diproses oleh algoritma Machine Learning. Teknik ini dilakukan dengan membuat kolom baru berdasarkan kategori yang tersedia dan memberikan nilai 1 atau 0 sesuai dengan kategori setiap data. Almajid (2022) menerapkan One-Hot Encoding sebagai salah satu teknik preprocessing dalam proses klasifikasi. Pada praktikum ini, teknik tersebut digunakan untuk mengubah kolom map menjadi beberapa kolom numerik.

2.4.4 Standardisasi Data

Standardisasi merupakan proses menyamakan skala nilai antarfitur agar tidak terdapat fitur dengan rentang nilai yang terlalu besar dibandingkan fitur lainnya. Pada praktikum ini digunakan metode StandardScaler dengan rumus:

$$Z = \frac{X - \mu}{\sigma}$$

Keterangan:

Z = Nilai setelah standardisasi.

X = Nilai awal.

μ = Nilai rata-rata.

σ = Standar deviasi.

Standardisasi penting terutama pada algoritma KNN karena perhitungan jaraknya dapat dipengaruhi oleh perbedaan skala setiap fitur.

2.5 Pembagian Data Training dan Testing

Pembagian data merupakan proses memisahkan dataset menjadi dua bagian, yaitu data training dan data testing. Data training digunakan untuk melatih model agar dapat mengenali pola dari dataset, sedangkan data testing digunakan untuk menguji kemampuan model dalam melakukan prediksi terhadap data yang belum digunakan selama pelatihan.

Pada praktikum ini digunakan perbandingan 75% data training dan 25% data testing. Dari total 1.961 data setelah proses penghapusan duplikat, sebanyak 1.470 data digunakan untuk training dan 491 data digunakan untuk testing.

2.6 Algoritma K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) merupakan algoritma Machine Learning yang digunakan untuk melakukan klasifikasi berdasarkan kedekatan jarak antara data yang akan diprediksi dengan data yang sudah memiliki label. Zhang et al. (2022) menjelaskan bahwa KNN merupakan metode klasifikasi yang menentukan kategori suatu data berdasarkan sejumlah tetangga terdekat.

Cara kerja KNN dimulai dengan menghitung jarak antara data baru dengan seluruh data training. Selanjutnya, sistem memilih sejumlah tetangga terdekat berdasarkan nilai K yang telah ditentukan. Kategori yang paling banyak muncul dari tetangga tersebut akan menjadi hasil prediksi.

2.6.1 Euclidean Distance

Salah satu metode yang digunakan untuk menghitung jarak pada algoritma KNN adalah Euclidean Distance. Metode ini menghitung jarak antara dua data berdasarkan perbedaan nilai pada setiap fitur. Rumusnya adalah:

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Keterangan:

d(x,y) = Jarak antara dua data.

x_i = Nilai fitur data pertama.

y_i = Nilai fitur data kedua.

n = Jumlah fitur.

Semakin kecil jarak yang diperoleh, semakin dekat hubungan antara kedua data tersebut.

2.6.2 Penentuan Nilai K

Nilai K merupakan jumlah tetangga terdekat yang digunakan untuk menentukan hasil klasifikasi. Pemilihan nilai K dapat memengaruhi hasil prediksi karena setiap nilai memiliki jumlah tetangga yang berbeda. Wang dan Zhang (2022) menjelaskan bahwa pemilihan jumlah tetangga merupakan salah satu faktor yang memengaruhi performa KNN.

Dalam praktikum ini dilakukan pengujian menggunakan dua nilai K, yaitu K=7 dan K=15. Selain itu, dilakukan pencarian nilai K terbaik dengan mencoba nilai 1 sampai 19 menggunakan Cross Validation untuk mengetahui nilai yang menghasilkan rata-rata akurasi tertinggi.

2.7 Algoritma Naive Bayes

Naive Bayes merupakan algoritma klasifikasi yang menggunakan konsep probabilitas berdasarkan Teorema Bayes untuk menentukan kemungkinan suatu data masuk ke dalam kategori tertentu. Algoritma ini memiliki asumsi bahwa setiap fitur bersifat independen atau tidak saling bergantung ketika kelas target sudah diketahui.

Rumus Teorema Bayes adalah:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Keterangan:

P(A|B) = Probabilitas A setelah diketahui B.

P(B|A) = Probabilitas B jika A terjadi.

P(A) = Probabilitas awal A.

P(B) = Probabilitas terjadinya B.

Naive Bayes menentukan kategori berdasarkan nilai probabilitas yang paling tinggi. Dalam praktikum ini digunakan Gaussian Naive Bayes untuk memprediksi kemenangan tim CT atau T berdasarkan fitur yang terdapat dalam dataset.

2.7.1 Gaussian Naive Bayes

Gaussian Naive Bayes merupakan salah satu jenis Naive Bayes yang digunakan untuk data numerik dengan asumsi bahwa nilai fitur mengikuti distribusi normal atau Gaussian. Perhitungan probabilitas dilakukan berdasarkan nilai rata-rata dan varians setiap fitur pada masing-masing kelas.

Rumus distribusi Gaussian adalah:

$$P(x|C) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Keterangan:

P(x|C) = Probabilitas fitur x pada kelas C.

x = Nilai fitur.

μ = Nilai rata-rata.
σ^2 = Varians.

Dalam praktikum ini, Gaussian Naive Bayes digunakan sebagai metode pembanding terhadap algoritma KNN untuk mengetahui perbedaan hasil klasifikasi dari kedua algoritma.

2.8 Cross Validation

Cross Validation merupakan teknik evaluasi yang digunakan untuk mengukur kemampuan model dengan membagi dataset menjadi beberapa bagian atau fold. Salah satu metode yang digunakan adalah K-Fold Cross Validation. Pada metode ini, data dibagi menjadi beberapa bagian dan setiap bagian digunakan secara bergantian sebagai data validasi.

Dalam praktikum ini digunakan 5-Fold Cross Validation untuk menguji nilai K pada algoritma KNN dari 1 sampai 19. Setiap nilai K diuji sebanyak lima kali, kemudian hasil akurasinya dirata-ratakan. Tujuannya adalah untuk mengetahui nilai K yang memberikan performa terbaik berdasarkan hasil pengujian. Arisandi et al. (2022) juga menerapkan K-Fold Cross Validation dalam pengujian model klasifikasi Naive Bayes.

2.9 Evaluasi Model Klasifikasi

Evaluasi model merupakan proses untuk mengetahui seberapa baik algoritma dalam melakukan klasifikasi terhadap data testing. Pada praktikum ini digunakan beberapa metrik evaluasi, yaitu accuracy, precision, recall, dan F1-score.

2.9.1 Accuracy

Accuracy merupakan ukuran untuk mengetahui perbandingan jumlah prediksi yang benar dengan seluruh data yang diuji.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Semakin tinggi nilai accuracy, semakin banyak prediksi yang sesuai dengan kategori sebenarnya.

2.9.2 Precision

Precision digunakan untuk mengukur ketepatan model ketika memprediksi suatu data ke dalam kategori tertentu.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Nilai precision yang tinggi menunjukkan bahwa hasil prediksi positif memiliki tingkat ketepatan yang baik.

2.9.3 Recall

Recall digunakan untuk mengukur kemampuan model dalam mengenali seluruh data yang benar-benar termasuk ke dalam kategori tertentu.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Semakin tinggi nilai recall, semakin baik kemampuan model dalam menemukan data yang sesuai dengan kategori sebenarnya.

2.9.4 F1-Score

F1-Score merupakan nilai gabungan antara precision dan recall yang digunakan untuk melihat keseimbangan keduanya.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Penggunaan beberapa metrik evaluasi diperlukan agar performa model dapat dinilai secara lebih menyeluruh, tidak hanya berdasarkan nilai akurasi (Walters, 2022).

2.10 Confusion Matrix

Confusion Matrix merupakan tabel yang digunakan untuk membandingkan hasil prediksi model dengan kategori sebenarnya. Tabel ini terdiri dari empat komponen, yaitu True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN).

Melalui Confusion Matrix, dapat diketahui jumlah prediksi yang benar maupun kesalahan klasifikasi yang dilakukan oleh model. Dalam praktikum ini, konsep tersebut digunakan untuk memahami kemampuan model dalam membedakan kemenangan tim CT dan T.

2.11 Python dan Scikit-Learn

Python merupakan bahasa pemrograman yang banyak digunakan dalam pengolahan data dan pengembangan Machine Learning karena memiliki berbagai library yang mendukung proses analisis. Salah satu library yang digunakan adalah Scikit-Learn.

Scikit-Learn menyediakan berbagai fungsi untuk melakukan preprocessing, pembagian dataset, pembuatan model, dan evaluasi hasil klasifikasi. Beberapa fungsi yang digunakan dalam praktikum ini antara lain train_test_split(), StandardScaler(), KNeighborsClassifier(), GaussianNB(), cross_val_score(), accuracy_score(), dan classification_report().

Penggunaan library tersebut membantu proses pengerjaan praktikum menjadi lebih mudah dan terstruktur, mulai dari pengolahan dataset hingga membandingkan hasil klasifikasi algoritma KNN dan Naive Bayes.

BAB III
HASIL

3.1 Persiapan dan Pembacaan Dataset

Pada praktikum UTS ini dilakukan proses klasifikasi menggunakan algoritma K-Nearest Neighbors (KNN) dan Naive Bayes dengan menggunakan dataset Counter Strike 2. Dataset yang digunakan memiliki 2.000 baris data dan 14 kolom yang berisi informasi mengenai kondisi pertandingan, seperti waktu tersisa, skor masing-masing tim, map yang digunakan, kondisi kesehatan, armor, uang, serta tim yang memenangkan ronde (round_winner).

Cell 0 – Import Library

```
[1]: # nurfadillahAP_202431126

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Tahap pertama dilakukan dengan mengimpor beberapa library Python yang dibutuhkan selama proses pengolahan data. Library Pandas dan NumPy digunakan untuk membantu mengelola serta melakukan perhitungan data. Sementara itu, Matplotlib dan Seaborn digunakan untuk membuat visualisasi dalam bentuk grafik agar hasil analisis lebih mudah dipahami.

Cell 1 – Membaca Dataset

```
[2]: #nurfadillahAP_202431126

data = pd.read_csv('03_counter_strike_2.csv')
data
```

[2]:

	time_left	ct_score	t_score	map	bomb_planted	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets	round_winner
0	175.00	0.0	0.0	de_dust2	False	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0	CT
1	156.03	0.0	0.0	de_dust2	False	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0	CT
2	96.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
3	76.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
4	174.97	1.0	0.0	de_dust2	False	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0	CT
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1995	74.90	7.0	10.0	de_train	False	327.0	355.0	376.0	183.0	6350.0	13300.0	3.0	1.0	T
1996	31.23	7.0	10.0	de_train	True	227.0	152.0	276.0	183.0	5000.0	7900.0	2.0	1.0	T
1997	11.23	7.0	10.0	de_train	True	185.0	105.0	266.0	183.0	5300.0	4450.0	2.0	1.0	T
1998	174.91	7.0	11.0	de_train	False	500.0	500.0	55.0	0.0	14900.0	32000.0	0.0	0.0	T
1999	114.90	7.0	11.0	de_train	False	500.0	500.0	355.0	500.0	650.0	6150.0	1.0	5.0	CT

2000 rows × 14 columns

Pada tahap ini dataset Counter Strike 2 dibaca menggunakan fungsi pd.read_csv(). Dataset yang memiliki format CSV dimasukkan ke dalam variabel data. Setelah berhasil dibaca, seluruh isi dataset ditampilkan untuk memastikan bahwa data sudah berhasil dimuat dan dapat digunakan untuk proses analisis selanjutnya.

Cell 2 – Menampilkan Data Awal

```
[3]: # nurfadillahAP_202431126

data.head()
```

[3]:

	time_left	ct_score	t_score	map	bomb_planted	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets	round_winner
0	175.00	0.0	0.0	de_dust2	False	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0	CT
1	156.03	0.0	0.0	de_dust2	False	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0	CT
2	96.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
3	76.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
4	174.97	1.0	0.0	de_dust2	False	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0	CT

Cell ini digunakan untuk menampilkan lima baris pertama dari dataset menggunakan fungsi head(). Tujuannya adalah untuk melihat gambaran awal data, mengetahui nama setiap kolom, serta memahami informasi yang tersedia dalam dataset sebelum dilakukan proses pengolahan lebih lanjut.

Cell 3 – Melihat Informasi Dataset

```
[4]: # nurfadillahAP_202431126

data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 14 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   time_left      2000 non-null   float64
 1   ct_score       2000 non-null   float64
 2   t_score        2000 non-null   float64
 3   map            2000 non-null   str
 4   bomb_planted   2000 non-null   bool
 5   ct_health      2000 non-null   float64
 6   t_health       2000 non-null   float64
 7   ct_armor       2000 non-null   float64
 8   t_armor        2000 non-null   float64
 9   ct_money       2000 non-null   float64
 10  t_money        2000 non-null   float64
 11  ct_helmets     2000 non-null   float64
 12  t_helmets      2000 non-null   float64
 13  round_winner   2000 non-null   str
dtypes: bool(1), float64(11), str(2)
memory usage: 224.8 KB
```

Pada tahap ini digunakan fungsi info() untuk mengetahui informasi mengenai dataset. Berdasarkan hasil output, dataset memiliki 2.000 baris dan 14 kolom. Seluruh kolom memiliki 2.000 data yang terisi sehingga tidak ditemukan data kosong pada tahap pemeriksaan awal. Informasi ini juga menunjukkan tipe data dari setiap kolom yang nantinya digunakan dalam proses preprocessing.

Cell 4 – Identifikasi Fitur Numerik

```
[5]: #nurfadillahAP_202431126

numerical_features = [feature for feature in data.columns if data[feature].dtypes != 'O']

print('Jumlah fitur numerik = ', len(numerical_features))
print('Daftar fitur numerik = ', numerical_features)

display(data[numerical_features].head())
```

```
Jumlah fitur numerik =  14
Daftar fitur numerik =  ['time_left', 'ct_score', 't_score', 'map', 'bomb_planted', 'ct_health', 't_health', 'ct_armor', 't_armor', 'ct_money', 't_money', 'ct_helmets', 't_helmets', 'round_winner']
```

	time_left	ct_score	t_score	map	bomb_planted	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets	round_winner
0	175.00	0.0	0.0	de_dust2	False	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0	CT
1	156.03	0.0	0.0	de_dust2	False	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0	CT
2	96.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
3	76.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
4	174.97	1.0	0.0	de_dust2	False	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0	CT

Cell ini digunakan untuk mengidentifikasi fitur yang dianggap memiliki tipe data numerik. Proses dilakukan dengan memeriksa tipe data setiap kolom, kemudian menampilkan jumlah dan nama fitur yang terdeteksi. Hasil output menunjukkan terdapat 14 kolom yang teridentifikasi sebagai fitur numerik. Namun, hasil tersebut juga memasukkan kolom map dan round_winner, sehingga identifikasi tipe data perlu diperhatikan kembali sebelum masuk ke tahap pemodelan.

Cell 5 – Identifikasi Fitur Kategorikal

```
[6]: #nurfadillahAP_202431126

categorical_features = [feature for feature in data.columns if data[feature].dtype == 'O']

print('Jumlah fitur categorical = ', len(categorical_features))
print('Daftar fitur categorical = ', categorical_features)

display(data[categorical_features].head())
```

```
Jumlah fitur categorical =  0
Daftar fitur categorical =  []
```

0
1
2
3
4

Pada cell ini dilakukan identifikasi terhadap fitur yang memiliki tipe data kategorikal atau berbentuk teks. Hasil output menunjukkan terdapat 0 fitur kategorikal berdasarkan pemeriksaan yang digunakan. Meskipun demikian, kolom map dan round_winner sebenarnya berisi kategori berbentuk teks. Oleh karena itu, pengubahan data kategorikal tetap dilakukan pada tahap preprocessing agar dapat digunakan oleh algoritma klasifikasi.

Cell 6 – Statistik Deskriptif

```
[7]: #nurfadillahAP_202431126

     data.describe()
```

[7]:

	time_left	ct_score	t_score	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets
count	2000.000000	2000.000000	2000.00000	2000.000000	2000.000000	2000.000000	2000.00000	2000.000000	2000.000000	2000.000000	2000.000000
mean	95.630205	6.840000	6.28850	406.170500	402.987500	306.749000	309.54450	7867.350000	12377.600000	1.821500	3.009500
std	53.734491	4.233599	4.19059	137.616258	137.699186	172.857221	168.68957	9017.491838	12209.581798	1.774884	1.960952
min	0.050000	0.000000	0.00000	2.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
25%	54.910000	3.000000	3.00000	315.750000	328.750000	188.000000	190.00000	1200.000000	1700.000000	0.000000	1.000000
50%	94.910000	7.000000	6.00000	500.000000	500.000000	300.000000	362.00000	4000.000000	9075.000000	1.000000	4.000000
75%	156.425000	10.000000	10.00000	500.000000	500.000000	488.000000	467.00000	11762.500000	19250.000000	3.000000	5.000000
max	175.000000	15.000000	15.00000	500.000000	500.000000	500.000000	500.00000	58050.000000	67700.000000	5.000000	5.000000

Pada tahap ini digunakan fungsi describe() untuk menampilkan ringkasan statistik dari fitur numerik. Hasilnya mencakup jumlah data, nilai rata-rata, standar deviasi, nilai minimum, kuartil, dan nilai maksimum. Dari hasil tersebut dapat diketahui gambaran umum kondisi data, seperti rata-rata kesehatan tim CT sebesar 406,17 dan tim T sebesar 402,99. Informasi ini membantu memahami karakteristik dataset sebelum dilakukan proses klasifikasi.

3.2 Pemeriksaan dan Visualisasi Data

Cell 7 – Mengecek Data Kosong

```
[8]: # nurfadillahAP_202431126
     data.isnull().sum()
```

```
[8]: time_left       0
     ct_score        0
     t_score         0
     map             0
     bomb_planted    0
     ct_health       0
     t_health        0
     ct_armor        0
     t_armor         0
     ct_money        0
     t_money         0
     ct_helmets      0
     t_helmets       0
     round_winner    0
     dtype: int64
```

Cell ini digunakan untuk memeriksa apakah terdapat data kosong atau missing value pada setiap kolom menggunakan fungsi isnull().sum(). Berdasarkan hasil output, seluruh kolom menunjukkan nilai 0. Artinya, tidak ditemukan data kosong pada dataset sehingga tidak diperlukan proses pengisian atau penghapusan data akibat missing value.

Cell 8 – Menghapus Data Duplikat

```
[9]: #nurfadillahAP_202431126

print("Jumlah data duplikat:", data.duplicated().sum())

data = data.drop_duplicates()

print("Jumlah data setelah menghapus duplikat:", len(data))
```

```
Jumlah data duplikat: 39
Jumlah data setelah menghapus duplikat: 1961
```

Pada tahap ini dilakukan pemeriksaan terhadap data yang memiliki isi sama menggunakan fungsi duplicated(). Hasil pemeriksaan menunjukkan terdapat 39 baris data duplikat. Selanjutnya, fungsi drop_duplicates() digunakan untuk menghapus data tersebut. Setelah proses selesai, jumlah dataset berkurang dari 2.000 menjadi 1.961 baris. Penghapusan ini dilakukan agar data yang digunakan tidak memiliki baris yang berulang.

Cell 9 – Menghitung Distribusi Round Winner

```
[10]: #nurfadillahAP_202431126
data['round_winner'].value_counts()
```

```
[10]: round_winner
T     1156
CT     805
Name: count, dtype: int64
```

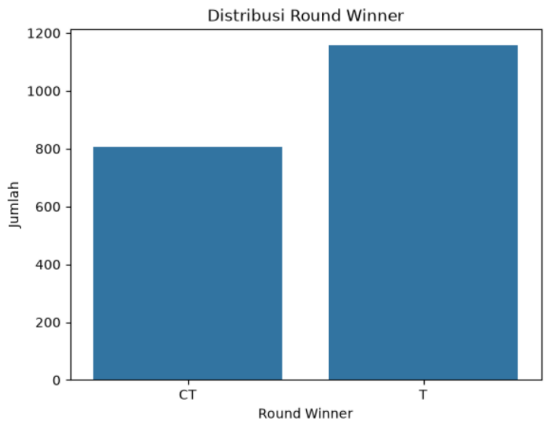
Cell ini digunakan untuk menghitung jumlah kemenangan masing-masing tim berdasarkan kolom round_winner. Hasil output menunjukkan bahwa tim T memperoleh 1.156 kemenangan, sedangkan tim CT memperoleh 805 kemenangan. Hasil tersebut memperlihatkan bahwa jumlah kemenangan tim T lebih banyak dibandingkan tim CT dalam dataset yang digunakan.

Cell 10 – Visualisasi Distribusi Round Winner

```
[11]: #nurfadillahAP_202431126

sns.countplot(x='round_winner', data=data)

plt.title('Distribusi Round Winner')
plt.xlabel('Round Winner')
plt.ylabel('Jumlah')
plt.show()
```



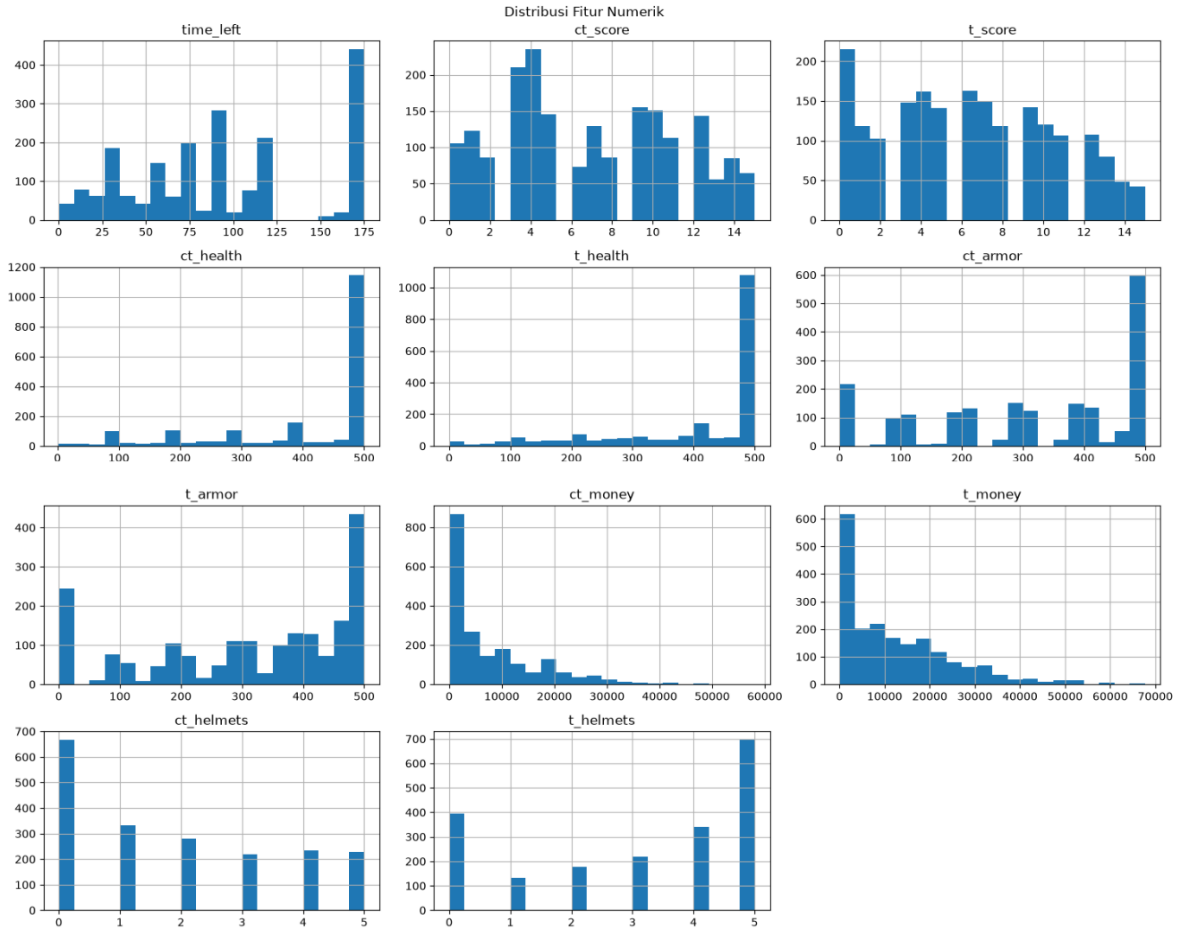
Pada tahap ini dibuat grafik batang menggunakan library Seaborn untuk menampilkan perbandingan jumlah kemenangan tim CT dan T. Sumbu X menunjukkan kategori tim yang memenangkan ronde, sedangkan sumbu Y menunjukkan jumlah kemenangan. Grafik ini mempermudah dalam melihat perbedaan jumlah kemenangan kedua tim berdasarkan hasil perhitungan sebelumnya.

Cell 11 – Visualisasi Distribusi Fitur Numerik

```
[12]: #nurfadillahAP_202431126

data.hist(figsize=(15, 12), bins=20)

plt.suptitle('Distribusi Fitur Numerik')
plt.tight_layout()
plt.show()
```



Cell ini digunakan untuk membuat histogram dari fitur-fitur numerik menggunakan fungsi hist(). Grafik dibuat dengan ukuran 15 × 12 dan menggunakan 20 bins agar penyebaran data terlihat lebih jelas. Melalui visualisasi ini dapat diketahui pola distribusi dari setiap fitur, seperti waktu pertandingan, skor, kesehatan, armor, dan jumlah uang yang dimiliki masing-masing tim.

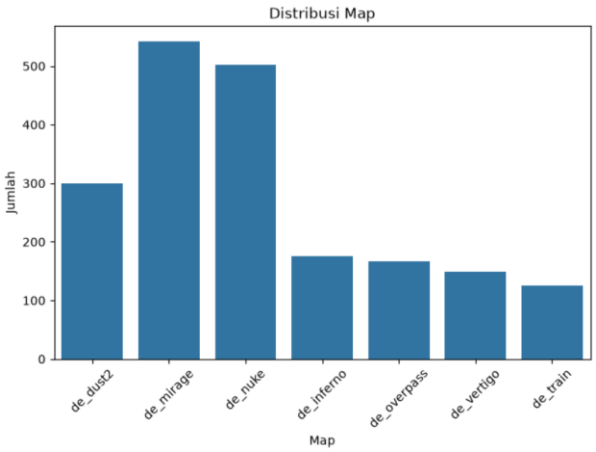
Cell 12 – Visualisasi Distribusi Map

Pada tahap ini dibuat grafik batang untuk mengetahui jumlah pertandingan berdasarkan map yang digunakan dalam Counter Strike 2. Grafik menggunakan kolom map sebagai sumbu X dan jumlah data sebagai sumbu Y. Label pada sumbu X juga diputar sebesar 45 derajat agar nama setiap map lebih mudah dibaca dan tidak saling bertumpuk.

```
[13]: #nurfadillahAP_202431126
plt.figure(figsize=(8, 5))

sns.countplot(x='map', data=data)

plt.title('Distribusi Map')
plt.xlabel('Map')
plt.ylabel('Jumlah')
plt.xticks(rotation=45)
plt.show()
```



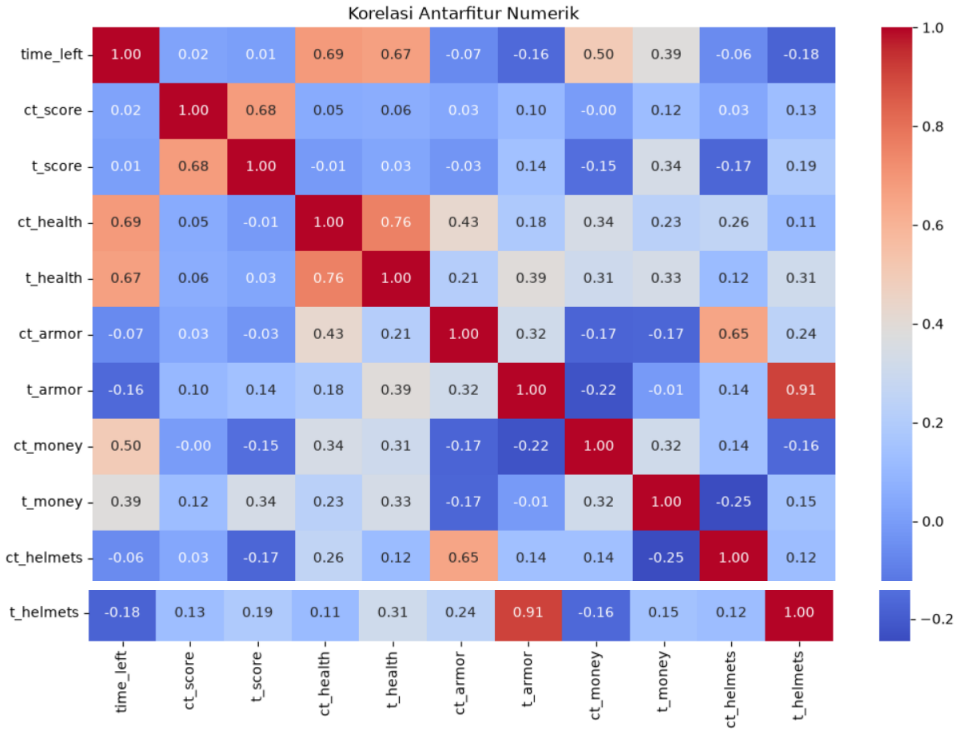
Cell 13 – Heatmap Korelasi Antarfitur

```
[14]: # nurfadillahAP_202431126

plt.figure(figsize=(12, 8))

sns.heatmap(
    data.select_dtypes(include=np.number).corr(),
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Korelasi Antarfitur Numerik')
plt.show()
```



Cell ini digunakan untuk membuat visualisasi heatmap yang menunjukkan hubungan atau korelasi antarfitur numerik. Perhitungan dilakukan menggunakan fungsi corr(), kemudian hasilnya ditampilkan dalam bentuk warna dan angka. Nilai korelasi positif menunjukkan hubungan searah, sedangkan nilai negatif menunjukkan hubungan berlawanan arah. Visualisasi ini membantu mengetahui hubungan antarvariabel yang terdapat dalam dataset.

3.3 Preprocessing Data

Cell 14 – Memisahkan Data Fitur dan Target

```
[15]: # nurfadillahAP_202431126

x = data.drop(['round_winner'], axis=1)
y = data['round_winner']

x.head()
```

[15]:

	time_left	ct_score	t_score	map	bomb_planted	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets
0	175.00	0.0	0.0	de_dust2	False	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0
1	156.03	0.0	0.0	de_dust2	False	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0
2	96.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0
3	76.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0
4	174.97	1.0	0.0	de_dust2	False	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0

Pada tahap ini dataset dibagi menjadi dua bagian, yaitu variabel X dan variabel Y. Variabel X berisi seluruh fitur yang digunakan sebagai bahan prediksi, sedangkan variabel Y berisi kolom round_winner sebagai target yang ingin diprediksi. Pemisahan ini dilakukan agar model dapat mempelajari hubungan antara kondisi pertandingan dengan hasil kemenangan tim.

Cell 15 – Menampilkan Data Target

```
[16]: #nurfadillahAP_202431126
      y.head()
```

```
[16]: 0    CT
      1    CT
      2    CT
      3    CT
      4    CT
      Name: round_winner, dtype: str
```

Cell ini digunakan untuk menampilkan lima data pertama dari variabel Y. Hasil output menunjukkan kategori target berupa CT dan T. Kedua kategori tersebut nantinya akan digunakan oleh algoritma KNN dan Naive Bayes untuk mempelajari pola serta menentukan hasil prediksi pemenang ronde.

Cell 16 – Mengubah Data Kategorikal

```
[17]: #nurfadillahAP_202431126
      x = pd.get_dummies(x, columns=['map'], dtype=int)

      x['bomb_planted'] = x['bomb_planted'].astype(int)

      print(x.dtypes)
      x.head()
```

Pada tahap ini dilakukan proses preprocessing menggunakan metode One-Hot Encoding untuk mengubah kolom map yang awalnya berbentuk teks menjadi beberapa kolom numerik. Setiap map akan memiliki kolom tersendiri dengan nilai 0 dan 1. Selain itu, kolom bomb_planted yang awalnya berbentuk Boolean diubah menjadi tipe integer. Setelah proses ini selesai, seluruh fitur sudah berbentuk numerik sehingga dapat digunakan dalam proses pemodelan machine learning.

Outputnya:

```
time_left          float64
ct_score           float64
t_score            float64
bomb_planted         int64
ct_health          float64
t_health           float64
ct_armor           float64
t_armor            float64
ct_money           float64
t_money            float64
ct_helmets         float64
t_helmets          float64
map_de_dust2         int64
map_de_inferno       int64
map_de_mirage        int64
map_de_nuke          int64
map_de_overpass      int64
map_de_train         int64
map_de_vertigo       int64
dtype: object
```

[17]:

	time_left	ct_score	t_score	bomb_planted	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets	map_de_dust2	map_de_inferno	map
0	175.00	0.0	0.0	0	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0	1	0	
1	156.03	0.0	0.0	0	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0	1	0	
2	96.03	0.0	0.0	0	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	1	0	
3	76.03	0.0	0.0	0	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	1	0	
4	174.97	1.0	0.0	0	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0	1	0	

Cell 17 – Membagi Data Training dan Testing

[18]:

```
#nurfadillahAP_202431126

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=0
)
```

Cell ini digunakan untuk membagi dataset menjadi data training dan data testing menggunakan fungsi train_test_split(). Pembagian dilakukan dengan komposisi 75% untuk data training dan 25% untuk data testing. Data training digunakan untuk melatih model agar dapat mengenali pola dari dataset, sedangkan data testing digunakan untuk menguji kemampuan model dalam melakukan prediksi terhadap data yang belum digunakan selama pelatihan.

Cell 18 – Mengecek Pembagian Data

[19]:

```
# nurfadillahAP_202431126
print(f'x_train shape : {x_train.shape}')
print(f'x_test shape : {x_test.shape}')
```

```
x_train shape : (1470, 19)
x_test shape : (491, 19)
```

Pada tahap ini dilakukan pengecekan terhadap hasil pembagian data menggunakan atribut shape. Hasil output menunjukkan bahwa data training memiliki 1.470 baris dan 19 kolom, sedangkan data testing memiliki 491 baris dan 19 kolom. Dengan demikian, total data yang digunakan setelah proses penghapusan duplikat adalah 1.961 baris.

Cell 19 – Standardisasi Data

```
[20]: #nurfadillahAP_202431126

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaler.fit(x_train)

x_train_scaled = scaler.transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Cell ini digunakan untuk melakukan standardisasi data menggunakan metode StandardScaler. Tujuannya adalah menyamakan skala setiap fitur agar tidak ada fitur dengan rentang nilai besar yang terlalu memengaruhi perhitungan jarak. Proses standardisasi dilakukan dengan mengambil parameter dari data training, kemudian diterapkan pada data training dan testing. Tahap ini sangat penting terutama untuk algoritma KNN karena menggunakan perhitungan jarak antar data dalam menentukan hasil klasifikasi.

3.4 Penerapan Algoritma K-Nearest Neighbors (KNN)

Cell 20 – Membuat Model KNN (K=7)

```
[21]: # nurfadillahAP_202431126

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=7)

knn.fit(x_train_scaled, y_train)
```

[21]: KNeighborsClassifier

▸ Parameters

▸ Fitted attributes

[21]: KNeighborsClassifier

▾ Parameters

n_neighbors	7
weights	'uniform'
algorithm	'auto'
leaf_size	30
p	2
metric	'minkowski'
metric_params	None
n_jobs	None

▾ Fitted attributes

Name	Type	Value
classes_	ndarray[object](2,)	['CT','T']
effective_metric_	str	'eu...an'
effective_metric_params_	dict	{}
n_features_in_	int	19
n_samples_fit_	int	1470
outputs_2d_	bool	False

Pada tahap ini dibuat model klasifikasi menggunakan algoritma K-Nearest Neighbors dengan nilai K sebesar 7. Artinya, model akan mempertimbangkan tujuh tetangga terdekat untuk menentukan kategori hasil prediksi. Setelah model dibuat, proses pelatihan dilakukan menggunakan data training yang sudah melalui standardisasi. Model kemudian mempelajari pola dari data tersebut untuk digunakan dalam proses prediksi.

Cell 21 – Melakukan Prediksi KNN

```
[22]: #nurfadillahAP_202431126
      y_pred = knn.predict(x_test_scaled)

      y_pred
```

```
[22]: array(['CT', 'T', 'CT', 'T', 'CT', 'T', 'CT', 'CT', 'T', 'T', 'CT', 'CT',
       'CT', 'T', 'T', 'CT', 'CT', 'CT', 'T', 'T', 'CT', 'T', 'CT', 'CT',
       'T', 'CT', 'CT', 'CT', 'CT', 'T', 'T', 'T', 'T', 'CT', 'CT', 'T',
       'T', 'CT', 'T', 'CT', 'CT', 'T', 'CT', 'CT', 'T', 'T', 'CT', 'T',
       'T', 'CT', 'CT', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T',
       'T', 'T', 'CT', 'T', 'T', 'CT', 'T', 'T', 'T', 'T', 'CT', 'T', 'T',
       'T', 'CT', 'T', 'T', 'T', 'CT', 'T', 'T', 'CT', 'T', 'T', 'T', 'T',
       'CT', 'CT', 'CT', 'T', 'T', 'CT', 'CT', 'T', 'T', 'T', 'CT', 'T',
       'CT', 'T', 'T', 'CT', 'CT', 'T', 'CT', 'T', 'CT', 'T', 'T', 'CT',
       'T', 'T', 'CT', 'CT', 'CT', 'T', 'T', 'CT', 'T', 'T', 'T', 'CT',
       'CT', 'T', 'CT', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'CT',
       'CT', 'CT', 'T', 'CT', 'T', 'T', 'CT', 'CT', 'T', 'T', 'CT', 'T',
       'T', 'CT', 'CT', 'CT', 'T', 'T', 'T', 'T', 'T', 'T', 'CT', 'CT',
       'T', 'T', 'CT', 'T', 'T', 'T', 'T', 'CT', 'T', 'CT', 'T', 'T', 'T',
       'T', 'CT', 'T', 'CT', 'T', 'CT', 'CT', 'T', 'CT', 'T', 'CT', 'T',
       'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'CT', 'CT', 'CT',
       'CT', 'CT', 'T', 'CT', 'CT', 'T', 'T', 'T', 'CT', 'T', 'CT', 'T',
       'T', 'CT', 'T', 'T', 'T', 'CT', 'T', 'T', 'CT', 'CT', 'CT', 'T',
       'CT', 'CT', 'T', 'CT', 'T', 'T', 'CT', 'T', 'CT', 'CT', 'T', 'T',
       'T', 'CT', 'T', 'T', 'T', 'T', 'CT', 'T', 'T', 'CT', 'CT', 'CT',
       'CT', 'T', 'CT', 'T', 'T', 'CT', 'T', 'T', 'T', 'T', 'T', 'CT',
       'CT', 'CT', 'T', 'CT', 'CT', 'CT', 'T', 'T', 'T', 'CT', 'T', 'T',
       'T', 'T', 'T', 'CT', 'CT', 'T', 'T', 'CT', 'T', 'CT', 'CT', 'T',
       'T', 'CT', 'T', 'T', 'T', 'CT', 'T', 'T', 'CT', 'CT', 'T', 'T',
       'CT', 'CT', 'T', 'T', 'CT', 'CT', 'T', 'CT', 'T', 'T', 'T', 'CT',
       'T', 'T', 'T', 'CT', 'T', 'T', 'T', 'CT', 'T', 'CT', 'T', 'T', 'T',
       'T', 'T', 'T', 'T', 'CT', 'T', 'T', 'CT', 'T', 'CT', 'T', 'CT',
       'T', 'T', 'T', 'T', 'CT', 'T', 'T', 'CT', 'T', 'T', 'T', 'CT', 'T',
       'T', 'T', 'T', 'T', 'T', 'T', 'CT', 'T', 'T', 'T', 'T', 'CT', 'T',
       'CT', 'T', 'CT', 'T', 'CT', 'T', 'T', 'T', 'T', 'T', 'CT', 'T',
       'CT', 'CT', 'T', 'T', 'T', 'CT', 'CT', 'T', 'T', 'T', 'CT', 'CT',
       'CT', 'T', 'CT', 'T', 'T', 'T', 'CT', 'CT', 'T', 'T', 'CT', 'CT',
       'T', 'CT', 'T', 'T', 'T', 'CT', 'CT', 'T', 'T', 'CT', 'CT', 'T',
       'T', 'CT', 'CT', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'CT', 'T',
       'T', 'CT', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'T', 'CT', 'T', 'CT',
       'T', 'T', 'T', 'T', 'CT', 'T', 'T', 'T', 'CT', 'T', 'T', 'T', 'CT',
       'T', 'T', 'T', 'T', 'CT', 'T', 'T', 'T', 'CT', 'CT', 'T', 'T',
       'CT', 'T', 'T', 'T', 'T', 'CT', 'CT', 'CT', 'T', 'T', 'T', 'CT',
       'CT', 'T', 'CT', 'CT', 'T', 'T', 'CT', 'T', 'CT', 'T', 'T'],
      dtype=object)
```

Cell ini digunakan untuk melakukan prediksi menggunakan model KNN yang sudah dilatih sebelumnya. Prediksi dilakukan terhadap data testing menggunakan fungsi predict(). Hasil prediksi disimpan ke dalam variabel y_pred dan menghasilkan kategori CT maupun T. Hasil tersebut nantinya dibandingkan dengan data target sebenarnya untuk mengetahui seberapa tepat model dalam melakukan klasifikasi.

Cell 22 – Evaluasi Model KNN (K=7)

Pada tahap ini dilakukan evaluasi terhadap model KNN dengan nilai K=7 menggunakan data testing. Evaluasi dilakukan dengan menghitung nilai akurasi serta menampilkan classification report yang berisi precision, recall, f1-score, dan support.

[23]:
```
#nurfadillahAP_202431126

from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score

accuracy_k7 = accuracy_score(y_test, y_pred)

print('Akurasi KNN (K=7):', accuracy_k7)
print(classification_report(y_test, y_pred))
```

```
Akurasi KNN (K=7): 0.780040733197556
              precision    recall  f1-score   support

          CT       0.77      0.68      0.72       208
           T       0.78      0.86      0.82       283

    accuracy                           0.78       491
   macro avg       0.78      0.77      0.77       491
weighted avg       0.78      0.78      0.78       491
```

Berdasarkan hasil pengujian, model KNN dengan K=7 memperoleh akurasi sebesar **78,00%**. Nilai precision untuk tim CT sebesar 0,77 dan tim T sebesar 0,78. Sementara itu, nilai recall tim CT sebesar 0,68 dan tim T sebesar 0,86. Hasil ini menunjukkan bahwa model memiliki kemampuan prediksi yang cukup baik, terutama dalam mengenali kemenangan tim T.

Cell 23 – Membuat Model KNN (K=15)

[24]:
```
#nurfadillahAP_202431126

K_TARGET = 15

model_k15 = KNeighborsClassifier(n_neighbors=K_TARGET)

model_k15.fit(x_train_scaled, y_train)

y_pred_k15 = model_k15.predict(x_test_scaled)

accuracy_k15 = accuracy_score(y_test, y_pred_k15)

print(f"Nilai K yang digunakan: {K_TARGET}")
print(f"Akurasi Model KNN pada data test: {accuracy_k15:.4f}")
```

```
Nilai K yang digunakan: 15
Akurasi Model KNN pada data test: 0.7597
```

Pada cell ini dilakukan pengujian menggunakan model KNN kedua dengan nilai K sebesar 15. Tujuannya adalah untuk mengetahui apakah perubahan jumlah tetangga terdekat dapat memengaruhi hasil akurasi model. Model dilatih menggunakan data training yang sama, kemudian digunakan untuk melakukan prediksi terhadap data testing. Berdasarkan hasil pengujian, model KNN dengan K=15 memperoleh akurasi sebesar **75,97%**. Hasil tersebut lebih rendah dibandingkan model KNN dengan K=7, sehingga pada pengujian ini penggunaan K=7 memberikan hasil yang lebih baik.

Cell 24 – Mencari Nilai K Terbaik

[25]:
```
#nurfadillahAP_202431126
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

k_values = range(1, 20)
accuracies = []

for k in k_values:
    model = make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=k)
    )

    scores = cross_val_score(
        model, x_train, y_train, cv=5, scoring='accuracy'
    )

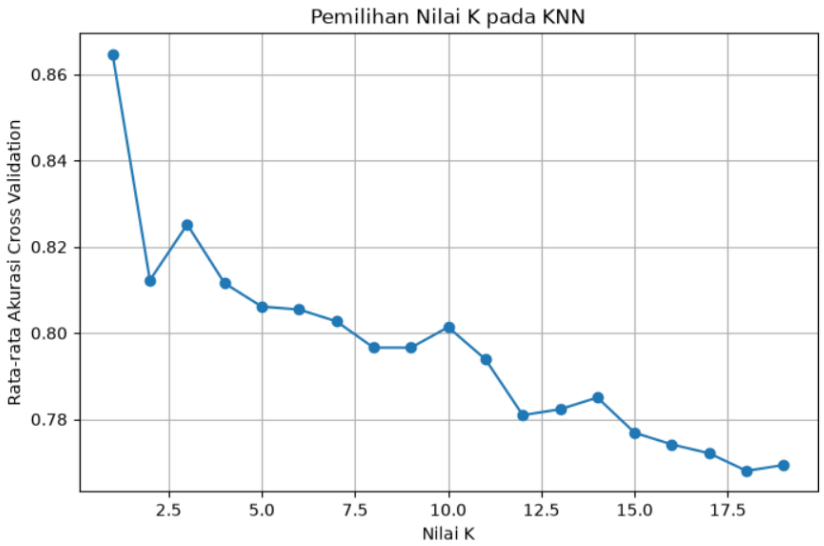
    accuracies.append(scores.mean())

plt.figure(figsize=(8, 5))

plt.plot(k_values, accuracies, marker='o')

plt.xlabel('Nilai K')
plt.ylabel('Rata-rata Akurasi Cross Validation')
plt.title('Pemilihan Nilai K pada KNN')
plt.grid(True)
plt.show()

print("Nilai K terbaik:", list(k_values)[np.argmax(accuracies)])
```



Pada tahap ini dilakukan pencarian nilai K terbaik menggunakan metode Cross Validation dengan mencoba nilai K dari 1 sampai 19. Pengujian dilakukan menggunakan 5-Fold Cross Validation, yaitu membagi data training menjadi lima bagian untuk mendapatkan rata-rata hasil akurasi. Setiap nilai K diuji menggunakan model KNN yang sudah digabungkan dengan proses standardisasi. Hasil pengujian ditampilkan dalam bentuk grafik untuk memperlihatkan perubahan akurasi pada setiap nilai K. Berdasarkan hasil output, nilai K terbaik yang diperoleh adalah **K=1**. Namun, nilai tersebut belum digunakan kembali untuk membangun model KNN baru pada perbandingan akurasi akhir.

3.5 Penerapan Algoritma Naive Bayes

Cell 25 – Membuat Model Naive Bayes

```
[26]: #nurfadillahAP_202431126

from sklearn.naive_bayes import GaussianNB

model_nb = GaussianNB()

model_nb.fit(x_train_scaled, y_train)

y_pred_nb = model_nb.predict(x_test_scaled)

accuracy_nb = accuracy_score(y_test, y_pred_nb)

print('Akurasi Naive Bayes:', accuracy_nb)
print(classification_report(y_test, y_pred_nb))
```

```
Akurasi Naive Bayes: 0.7128309572301426
              precision    recall  f1-score   support

          CT       0.66      0.66      0.66       208
           T       0.75      0.75      0.75       283

    accuracy                           0.71       491
   macro avg       0.71      0.71      0.71       491
weighted avg       0.71      0.71      0.71       491
```

Pada tahap ini dibuat model klasifikasi menggunakan algoritma Gaussian Naive Bayes. Algoritma ini menggunakan perhitungan probabilitas untuk menentukan kemungkinan suatu data masuk ke dalam kategori tertentu berdasarkan fitur yang dimiliki. Model dilatih menggunakan data training yang sudah melalui proses standardisasi, kemudian digunakan untuk memprediksi data testing.

Berdasarkan hasil pengujian, model Naive Bayes memperoleh akurasi sebesar **71,28%**. Hasil classification report menunjukkan bahwa nilai precision dan recall untuk tim CT masing-masing sebesar 0,66, sedangkan untuk tim T masing-masing sebesar 0,75. Hasil tersebut menunjukkan bahwa model Naive Bayes mampu melakukan klasifikasi, tetapi akurasinya masih lebih rendah dibandingkan kedua model KNN yang diuji sebelumnya.

3.6 Perbandingan dan Evaluasi Hasil Model

Cell 26 – Membandingkan Hasil Akurasi

```
[27]: #nurfadillahAP_202431126
hasil = pd.DataFrame({
    'Algoritma': ['KNN (K=7)', 'KNN (K=15)', 'Naive Bayes'],
    'Akurasi': [accuracy_k7, accuracy_k15, accuracy_nb]
})

hasil['Akurasi (%)'] = hasil['Akurasi'] * 100

hasil
```

[27]:

	Algoritma	Akurasi	Akurasi (%)
0	KNN (K=7)	0.780041	78.004073
1	KNN (K=15)	0.759674	75.967413
2	Naive Bayes	0.712831	71.283096

Pada tahap ini dilakukan perbandingan hasil akurasi dari tiga model yang telah dibuat, yaitu KNN dengan K=7, KNN dengan K=15, dan Naive Bayes. Hasil akurasi dari masing-masing model dimasukkan ke dalam DataFrame agar lebih mudah dibaca dan dibandingkan.

Nilai akurasi yang awalnya berbentuk desimal juga dikalikan 100 untuk mendapatkan hasil dalam bentuk persentase.

Berdasarkan hasil output, diperoleh perbandingan sebagai berikut:

Tabel 3.1 Perbandingan Akurasi Model

Algoritma	Akurasi
KNN (K=7)	78,00%
KNN (K=15)	75,97%
Naive Bayes	71,28%

Dari hasil tersebut, dapat diketahui bahwa KNN dengan K=7 memperoleh akurasi paling tinggi, yaitu 78,00%. Sementara itu, KNN dengan K=15 memperoleh akurasi sebesar 75,97% dan Naive Bayes memperoleh akurasi sebesar 71,28%.

Cell 27 – Visualisasi Perbandingan Akurasi

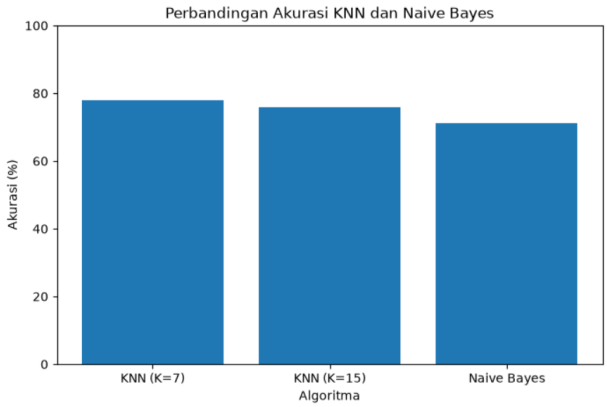
```
[28]: #nurfadillahAP_202431126

plt.figure(figsize=(8, 5))

plt.bar(hasil['Algoritma'], hasil['Akurasi (%)'])

plt.ylabel('Akurasi (%)')
plt.xlabel('Algoritma')
plt.title('Perbandingan Akurasi KNN dan Naive Bayes')
plt.ylim(0, 100)

plt.show()
```



Cell ini digunakan untuk menampilkan hasil perbandingan akurasi ketiga algoritma dalam bentuk grafik batang. Setiap batang menunjukkan nilai akurasi dari masing-masing model, sehingga perbedaan performanya dapat dilihat dengan lebih mudah. Berdasarkan grafik yang dihasilkan, KNN dengan K=7 memiliki batang paling tinggi dibandingkan kedua model lainnya. Hal ini menunjukkan bahwa model tersebut memberikan hasil klasifikasi terbaik berdasarkan pengujian pada data testing yang digunakan.

3.7 Pembahasan Hasil Praktikum

Berdasarkan seluruh proses pengujian yang telah dilakukan, algoritma K-Nearest Neighbors (KNN) dengan nilai K=7 memberikan hasil akurasi tertinggi dibandingkan KNN dengan K=15 dan Naive Bayes. Hal ini menunjukkan bahwa penggunaan tujuh tetangga terdekat pada dataset Counter Strike 2 menghasilkan prediksi yang lebih sesuai dengan data sebenarnya dibandingkan penggunaan lima belas tetangga terdekat.

Model KNN dengan K=15 memperoleh akurasi yang sedikit lebih rendah. Sementara itu, model Naive Bayes memperoleh akurasi sebesar 71,28%, sehingga berada di posisi ketiga dalam perbandingan model. Perbedaan hasil tersebut menunjukkan bahwa setiap algoritma memiliki cara kerja dan kemampuan yang berbeda dalam mengenali pola pada dataset.

Dari hasil praktikum ini dapat disimpulkan bahwa algoritma KNN dengan K=7 merupakan model dengan performa terbaik dari ketiga model yang diuji, dengan tingkat akurasi sebesar 78,00%. Meskipun demikian, hasil tersebut masih dapat dikembangkan melalui pengujian parameter dan proses preprocessing yang lebih optimal agar kemampuan klasifikasi menjadi lebih baik.

BAB IV
PENUTUP

4.1 Kesimpulan Landasan Teori

Berdasarkan landasan teori yang telah dibahas, dapat disimpulkan bahwa Machine Learning merupakan salah satu teknologi yang dapat digunakan untuk mengolah data, mengenali pola, dan menghasilkan prediksi berdasarkan informasi yang tersedia. Dalam proses klasifikasi, terdapat beberapa tahapan penting yang perlu dilakukan, mulai dari pengumpulan dataset, preprocessing, pembagian data training dan testing, hingga evaluasi model. Algoritma K-Nearest Neighbors (KNN) bekerja dengan menentukan kategori berdasarkan jarak terdekat antar data, sedangkan Naive Bayes menggunakan perhitungan probabilitas berdasarkan Teorema Bayes. Kedua algoritma tersebut memiliki cara kerja yang berbeda, tetapi sama-sama dapat digunakan untuk menyelesaikan permasalahan klasifikasi. Selain itu, penggunaan teknik preprocessing, Cross Validation, dan metrik evaluasi seperti accuracy, precision, recall, serta F1-score diperlukan untuk membantu menghasilkan model yang lebih baik dan mengetahui tingkat keberhasilan proses klasifikasi.

4.2 Kesimpulan Hasil Praktikum

Berdasarkan hasil praktikum UTS yang telah dilakukan menggunakan dataset Counter Strike 2, dapat disimpulkan bahwa proses klasifikasi berhasil dilakukan menggunakan algoritma KNN dan Naive Bayes. Tahapan praktikum dimulai dari membaca dataset, melakukan pemeriksaan dan pembersihan data, mengubah data kategorikal menjadi numerik, melakukan standardisasi, serta membagi dataset menjadi data training dan testing. Berdasarkan hasil pengujian, algoritma KNN dengan nilai K=7 memperoleh akurasi sebesar 78,00%, KNN dengan K=15 memperoleh akurasi sebesar 75,97%, sedangkan Gaussian Naive Bayes memperoleh akurasi sebesar 71,28%. Dari ketiga model tersebut, KNN dengan K=7 memberikan hasil akurasi paling tinggi sehingga menjadi model terbaik berdasarkan pengujian yang dilakukan. Selain itu, proses Cross Validation juga dilakukan untuk mencari nilai K terbaik dan menghasilkan K=1. Secara keseluruhan, praktikum ini memberikan pemahaman mengenai penerapan algoritma klasifikasi serta cara membandingkan performa model Machine Learning berdasarkan hasil prediksi dan evaluasi yang diperoleh.

DAFTAR PUSTAKA

1. Almajid, A. S. (2022). Multilayer Perceptron Optimization on Imbalanced Data Using SVM-SMOTE and One-Hot Encoding for Credit Card Default Prediction. Journal of Advances in Information Systems and Technology, 3(2), 67–74. https://doi.org/10.15294/jaist.v3i2.57061
2. Arisandi, R. R. R., Warsito, B., & Hakim, A. R. (2022). Aplikasi Naïve Bayes Classifier (NBC) pada klasifikasi status gizi balita stunting dengan pengujian K-Fold Cross Validation. Jurnal Gaussian, 11(1), 130–139. https://doi.org/10.14710/j.gauss.v11i1.33991
3. Cho, E., Chang, T. W., & Hwang, G. (2022). Data preprocessing combination to improve the performance of quality classification in the manufacturing process. Electronics, 11(3), 477. https://doi.org/10.3390/electronics11030477
4. Walters, W. P. (2022). Comparing classification models—a practical tutorial. Journal of Computer-Aided Molecular Design, 36, 381–389. https://doi.org/10.1007/s10822-021-00417-2
5. Wang, B., & Zhang, S. (2022). A new locally adaptive K-nearest centroid neighbor classification based on the average distance. Connection Science, 34(1), 2084–2107. https://doi.org/10.1080/09540091.2022.2088695
6. Zhang, X., Xiao, H., Gao, R., Zhang, H., & Wang, Y. (2022). K-nearest neighbors rule combining prototype selection and local feature weighting for classification. Knowledge-Based Systems, 243, 108451. https://doi.org/10.1016/j.knosys.2022.108451